Arctos Robotic Arm CAN Bus & ROS 2 Integration

Engineering Log

Bench log — Arctos CAN integration

Last updated 2026-09-03

Contents

About this log

This document is a running **engineering log** of bringing the Arctos arm up on CAN and ROS 2. It is not a standard operating procedure, and it should not be read as one: entries record what was tried on a given day, what the hardware actually did, and what was concluded — including the conclusions that later turned out to be wrong. Several entries exist specifically to stop a future session repeating a dead end, so a superseded finding is left in place with a note pointing at the entry that overturned it, rather than being deleted.

Entries appear in chronological order. Dates are given where the original entry recorded one; a few early entries did not, and no date has been invented for them.

The stable, procedural material — network bring-up, the joint/CAN-ID map, protocol errata, the per-joint bring-up steps and the Python stack — has been moved to the appendices, where it can be looked up without reading the history. The four reference documents in `docs/` (`Arctos_CAN_Interface`, `BASIC_CAN_COMMANDS`, `Nema_17_MKS42D_SOP` and `Nema_23_MKS57D_SOP`) hold the fuller reference treatment.

Caution: Read the newest entry on a joint before trusting an older one

This log contains findings that were later disproved. Joint C in particular was recorded on 2026-08-25 as having a fault "no CAN command can fix"; the 2026-09-03 entry shows that joint driving its load correctly. Always search forward for the most recent entry naming the joint you are working on.

---

1 2026-08-25 — Joint C bring-up attempt: no holding torque

Picking this up again: Joint C is wired, powered, and communicating over CAN, but **does not produce holding torque under any configuration tried today**. This section records exactly what was tried and ruled out, so the next session doesn't repeat it.

1.1 Confirmed working

- CAN link is solid: Step 2's reliability test passed cleanly (100% response rate, 0 checksum failures, 0 encoder jitter across 200 samples).
- CAN termination jumper near the CAN_H/CAN_L terminals was confirmed present and left installed – correct, since this driver is currently one of only two nodes on the bus (CANable and this driver).
- Main motor power measured at 24.9 V DC at the driver – normal and expected for this driver family (not the 249.6 V initially misreported).
- All four motor phase wires confirmed firmly seated; the driver-to-motor connector was additionally unplugged and re-seated mid-session. Neither changed the outcome.
- The joint was reportedly calibrated previously. Calibration (`0x80 CALIBRATE`) was **not** re-run today because the motor is currently coupled to its gear train and cannot be decoupled right now – `motor_driver.cpp` explicitly warns calibration should be done unloaded, so re-running it under load was judged higher-risk than leaving it as-is.

1.2 CAN ID: moved during the session – always re-scan, never assume

`can_id_scan.py` first found Joint C responding at `0x01` (matching neither `arctos_controller.yaml`'s `0x06` nor the older docs' `0x03`). Partway through the session the CAN ID was changed on the driver's own panel to `0x06`, confirmed again by re-scanning – **do not assume either value going forward; run `can_id_scan.py` at the start of the next session before anything else.**

1.3 Ruled out: every CAN/panel-configurable setting tried so far

All of the following were tested (via direct CAN commands and/or the driver's own physical panel) with **no change in outcome** – `READ_ENABLE_STATE (0x3A)` sometimes read back `1` (enabled) and sometimes `0` depending on mode, but in every single case the shaft span completely freely by hand with no detectable holding torque:

- **Working mode**: `SR_vFOC` (value 5, set both via `0x82` over CAN and later on the panel directly), `CR_FOC`/`CR_vFOC` (value 2, set on the panel), and `CR_CLOSE` (value 1, set on the panel – this is what Joint B's panel is documented to use).
- **Working current**: the panel's stored default of 3000 mA (roughly 2.5× this motor's documented 1.2 A rating – flagged and corrected), and 1200–1600 mA (matching the documented rated current), both via `0x83` over CAN and via the panel.
- **CAN ID**: `0x01` and `0x06` (see above).

Important corroborating observation: after several enable/hold cycles at up to 3000 mA, **the motor showed no detectable warmth at all**. If current were actually reaching the coils, even without producing net rotation, resistive (I^2R) heating should be noticeable fairly quickly at that current. Its complete absence across every trial points away from a settings/calibration problem and toward an open or broken path between the driver's motor-output stage and the coils – something no CAN command can fix.

1.4 Not yet tried / next session

- **Phase resistance check**: with power off, measure resistance across each of the two motor phase-wire pairs directly at the driver-to-motor connector (a healthy phase for this motor class should read roughly 1–5 Ω; open/infinite indicates a broken phase). Not done today – no multimeter was available.
- **Component swap test**: temporarily connect Joint B's known-working motor to Joint C's driver (or Joint C's motor to Joint B's driver) to isolate whether the fault is in the driver or in the motor/cable. Not attempted today.
- **Live comparison against Joint B**: the written guide documents describing Joint B's setup (`arctos_CAN_ROS2_SOP_updated.tex`, the since-removed `arctos_joint_b_guide.tex`, the cheat sheet) all still describe Joint B's own motion as unconfirmed as of Aug 19–20, even though Joint B has since been confirmed to move – whatever actually fixed it isn't recorded in any document in this workspace. Check Joint B's driver's *actual current* panel settings directly (working mode, current, CANRSP) and compare against whatever Joint C ends up needing, rather than trusting the stale written docs.
- `SET_ENABLE_SETTINGS (0x85)` exists in `motor_types.hpp` but its payload format is undocumented anywhere in this project and was deliberately not guessed at. If the panel exposes an equivalent enable-pin/enable-source setting, check it there instead.

2 2026-08-27 — Joint B diagnostics and large-move test

Two days after the Joint C session above, Joint C's own troubleshooting was paused (multimeter phase-resistance check and driver/motor swap test still pending – see Section 1), and Joint B was connected and powered instead to use as a known-working reference.

2.1 Joint B: confirmed healthy, with real holding torque

`can_id_scan.py` confirmed Joint B at `0x05` (matching every document, unlike Joint C's repeated mismatches). `joint_reliability_test.py` came back clean: 100% response rate, 0 checksum failures, 1 raw count of jitter (noise floor) across 200 samples. Critically, Joint B's coils were found already enabled (`coils_enabled=True` before this session sent anything), and **the shaft genuinely resisted turning by hand** – real holding torque, unlike every attempt on Joint C.

2.2 Joint B's verified-working panel configuration

There is no `READ_WORKING_MODE` or `READ_CURRENT` opcode anywhere in `motor_types.hpp` – these can only be read from the driver's own physical panel, not queried over CAN. Read directly off Joint B's screen while it was confirmed holding torque: **This is a new working mode not yet**

Setting	Joint B's verified-working value
Working mode	`SR_CLOSE` (enum value 4)
Current	1600 mA (matches `motor_types.hpp`'s own `working_current{1600}` default exactly)
CANRSP	Enable

Table 1: Joint B live-verified panel configuration, 2026-08-27.

tried on Joint C – Joint C testing on 2026-08-25 covered `SR_vFOC` (5), `CR_vFOC`/`CR_FOC` (2), and `CR_CLOSE` (1), but not `SR_CLOSE` (4). Next Joint C session: set `SR_CLOSE` + 1600 mA + confirm CANRSP=Enable, then re-run the hold-enable test.

2.3 20-degree monitored move test on Joint B: real motion, but not reliably reproducible per-command

With Joint B confirmed healthy, a user-requested ∼20° joint move was run as 5 monitored steps of 4° each (Section D's absolute-vs-relative ambiguity for `0xF5` was never resolved, so the incremental approach from Section E.3 was used rather than a single blind large command). Starting position was ≈ 0.00, well inside the calibrated envelope (`home_position`=+12.87, `opposite_limit`=−24.23 per `arctos_controller.yaml`).

Result: Step 1 produced real motion in the wrong direction and wrong magnitude (commanded +4, actual −1.84), after which **Steps 2–5 produced essentially no further motion** (each < 0.001) despite each commanding a distinct new absolute target. No protection/stall flag was ever set; final position (−1.85) stayed safely inside the envelope. The incremental/monitored design caught this after a small, safe excursion rather than after a blind full-magnitude commit.

A follow-up test re-sent `ENABLE_MOTOR` immediately before every `0xF5` and added a 3-second pause between steps, using smaller 2 steps. **This produced real motion of very close to the commanded magnitude on 2 of 3 steps** (Step 1: commanded +2.0, actual +1.96; Step 3: commanded +2.0, actual +1.96) – a clear improvement over the first attempt, where only the very first command after enabling produced any real motion at all. **But Step 2 landed nowhere near its own commanded target**, instead landing within ∼13 raw counts of where the joint had been *two steps earlier* – even though the transmitted frame's position bytes were verified correct and distinct from the other two steps. This is not a simple sign-flip or absolute-vs-relative confusion; it looks like a command-queuing or timing race between consecutive `0xF5` frames, where a stale target sometimes wins over the freshly-sent one.

Caution: Do not trust a single `0xF5` command's outcome without reading the encoder back

Across both tests today, magnitude and direction were unpredictable often enough (3 of 8 total commanded steps across both runs landed far from their target) that **every `0xF5` command must be followed by an encoder read to confirm where the joint actually ended up** before commanding anything further, exactly as the incremental procedure in Section E.3 already does. No error or shaft-protection flag was set during any of these mismatches – the driver does not flag this condition on its own.

2.4 RELATIVE_POSITION (0xF4): a single move matched commanded magnitude almost exactly

A single 0xF4 command (delta +6173 raw counts, ≈ 2.0°) produced −1.98° of real motion – magnitude within 1% of commanded, direction flipped as expected from B_joint's inverted: true flag (these raw test scripts deliberately do not apply that flip themselves; see the caution in Section E's tiny-move script). This was the cleanest single-command result of the entire session.

But 3 consecutive 0xF4 commands (same delta, sent with only a single upfront ENABLE_MOTOR), run twice, produced almost no motion on any of the 3 steps both times (< 0.04° per step) – reproducing the same partial-execution pattern seen with 0xF5, just now confirmed on 0xF4 too. This is not an absolute-vs-relative distinction; both commands show the same symptom.

2.5 candump capture: the driver genuinely acknowledges "movement started" – but barely moves

A full unfiltered candump -tz can0 capture during a 3-step 0xF4 sequence resolved an open question: is our own polling code silently discarding relevant frames? No. Every F4 00 32 0A 00 18 1D 6A command received a real, checksummed F4 01 FA response – decoding status=1 against motor_driver.cpp's own convention for ABSOLUTE_POSITION (case 1: // Movement started), **the driver is telling us, correctly, that it started moving, every single time**. No other frames were ever sent besides our own read requests and this one ack. Meanwhile the raw encoder value crept by only ∼115 counts total across all three commands (each requesting 6173) – roughly 2% of one single step.

2.6 90-second and 60-second single-command watches: a genuine two-phase profile, not "just slow"

Watching a single 0xF4 command for much longer than the ∼5s used everywhere else in this project revealed the real shape of the response: a **real, fast burst** of motion in the first few seconds (−737 counts/s at speed=50,accel=10; −428 counts/s at a much gentler speed=15,accel=1), covering the large majority of whatever motion occurs, followed immediately by an **effective stop** – creeping at noise-floor level (∼0.3 counts/s) for the remaining ∼80+ seconds of each test, never approaching the full 6173-count target. Total captured: −2353 counts (38%) at the faster settings, −1301 counts (21%) at the gentler settings – **a much gentler ramp did not fix it, and if anything captured less of the move**, ruling out "acceleration too aggressive for the load" as the primary cause. No 0x3E shaft-protection flag was ever set in either test.

Caution: A real sound was heard at the burst-to-stop transition, on two separate occasions

The user reported hearing something at the point where motion stopped, on two separate test runs. **Physical inspection ruled out an external obstruction**: no visible obstruction, the bare motor shaft (checked before the gearbox) moves freely and does not catch, and **the arm segment itself was confirmed to visibly move** during at least one test (ruling out total gearbox/coupling slippage, since the encoder is mounted on the motor's own shaft *before* the gearbox and would not by itself prove the joint's output side actually moved). The combination points toward the motor **losing synchronization mid-move** (a stepper "chuff"/step-loss event) rather than a hard mechanical limit. **The motor was also reported "quite warm"** after this sequence of tests

at 1600 mA (documented rated current is 1.2 A) – testing was paused for a cool-down rather than continuing to add more energized time on top of that.

2.7 A promising lead: the real production system never sends one far-away target

Checking `arctos_interface.cpp`'s `write()` function directly: it only calls `setJointPosition()` when the commanded position changes by more than `position_tolerance_` – meaning in real operation, a `JointTrajectoryController` continuously publishes a smoothly interpolated, frequently-updated intermediate setpoint (at the controller's 100 Hz `update_rate`), and `write()` forwards each meaningfully-different point as a fresh CAN frame. **Every test in this entire session has instead sent one single far-away target and waited** – the opposite of how this driver has ever actually been exercised by the real stack. The burst-then-stop pattern is consistent with the driver executing some bounded internal motion profile per command and requiring a fresh, nearby target to keep advancing, which one-shot far-away commands never provide. Not yet tested: `joint_streamed_move_test.py` (added this session, not yet run) sends the total move as many small increments in rapid succession to test this directly.

2.8 A second promising lead: `POSITION_CONTROL (0xFD)`, not `0xF4/0xF5`, is what the project's own working jog code actually uses

`POSITION_CONTROL (0xFD)` is defined in `motor_types.hpp` but **never referenced anywhere in `motor_driver.cpp`** – the C++ ROS 2 driver only ever uses `0xF5`. It *is*, however, the command used by `scripts/set_zero_position.py`'s `relative_motion_by_pulses()`, called repeatedly in a loop from `find_home_position()`, `find_opposite_limit()`, and `manual_move_joint()` – i.e., exactly the repeated-small-jog pattern this session has been struggling to get right with `0xF4/0xF5`, and part of the one documented setup flow (`--init` in the `ros2_arctos` README) that this project actually relies on working. **Its frame layout is structurally different**: a direction bit (`0x80/0x00`) is OR'd into the top of the speed-high byte, and its position field is in **microstep pulses** (200 full steps/rev × 16 microsteps/step = 3200 pulses/motor-rev) rather than the 16384-count/rev encoder scale every other script in this document uses for `0xF4/0xF5`. Not yet tested: `joint_position_control_test.py` (added this session, matches `set_zero_position.py`'s exact byte-packing) is ready to try next session.

2.9 New reusable scripts added this session

- `joint_stepped_move_test.py` – generalized N-step monitored move (absolute or relative, `--mode`), safety-band checked against calibrated `home_position/opposite_limit`, optional `--re-enable-each-step`.
- `joint_streamed_move_test.py` – sends a total move as many rapid small `0xF4` increments instead of one command, to test the trajectory-streaming hypothesis above. Not yet run.
- `joint_position_control_test.py` – `0xFD`-based jog test matching `set_zero_position.py`'s exact pulse-scale and direction-bit convention. Not yet run.

2.10 Characterizing (not resolving) 0xFD: a real, deterministic success signal with a still-unknown trigger

joint_position_control_test.py (0xFD) revealed a protocol detail this whole investigation had been missing: **the driver sends an unsolicited two-frame status sequence per command** – an immediate FD 01 (status=1, "movement started", matching motor_driver.cpp's own convention) followed, sometimes, by a second FD 02 (status=2, almost certainly "movement complete") arriving up to ∼425 ms later. **Whether that second frame arrives correlates perfectly with whether the commanded magnitude is actually achieved** – confirmed by directly matching a raw candump capture against per-step results across an 8-step test: both steps that received FD 02 moved within 1% of their commanded 1°; all six steps that received only FD 01 moved less than 0.04° each, several essentially zero.

An initial 3-step test (re-enabling before every command) looked like a full resolution: individual steps of +0.025°, +1.974°, +0.998° summed to +2.997° against a 3.0° total commanded (99.9%), suggesting an incomplete command's shortfall carries forward into the next one. **This did not hold up when confirmed over a longer 8-step sequence**: steps 1–2 moved correctly (matching FD 02 frames confirmed via candump), step 3 partially, and steps 4–8 essentially stopped entirely (each receiving only FD 01) despite an identical fresh-enable-before-each-command pattern – total movement was 32% of commanded, not 99.9%. A follow-up run logging READ_ERROR (0x39) and READ_IO (0x34) (the latter never queried anywhere else in this project) at fine resolution around a fresh sequence found **the very first command of that run failed too** (only FD 01, negligible motion) – contradicting even the narrower "the first 1–2 commands always succeed" pattern.

Neither additional status opcode explained the difference: READ_IO (0x34) stayed completely constant ([52, 1, 58]) across every query in that run regardless of success or failure, and READ_ERROR (0x39)'s trailing bytes changed on nearly every single query whether or not real motion occurred – behavior more consistent with a free-running counter than a meaningful error/stall code. READ_SHAFT_PROTECTION_STATE (0x3E) never once left its baseline value across any test this session, successful or not.

A retry-on-FD-02 mechanism (joint_position_control_test.py, up to 4 retries/step, always re-enabling first) was then built and tested: in one run, **all 15 attempts across 3 steps got only FD 01, never a single** FD 02 – yet the joint still accumulated real, if uneven, motion across those retries. This broke the working theory that FD 02 cleanly gates real motion, and pointed toward something more fundamental than a CAN-protocol quirk.

2.11 ENABLE_SHAFT_PROTECTION (0x88): a promising lead that turned out to be a false-positive trap

The user reported that manually assisting the joint by hand let a stuck move complete – direct physical evidence pointing at genuine mechanical resistance. This led to checking whether ENABLE_SHAFT_PROTECTION (0x88) – the command that actually turns on the "Tracking Error Protection Stall" feature described all the way back in Section 1 of this document – had ever been sent. **It had not, anywhere in this project**: not in any test script this session, not in the real C++ arctos_hardware_interface, not in set_zero_position.py. Without it enabled, READ_SHAFT_PROTECTION_STATE (0x3E) reads its baseline value regardless of whether a real stall is occurring.

Sending 0x88 0x01 once, then repeating a 1° move, appeared to confirm it immediately: **the very first move afterward tripped the flag** (status 0 → 1), with the joint having moved only 0.0017° before the driver locked itself down (requiring a power-cycle of its main supply to clear, per

Section 1). Raising current to 2000 mA and separately trying a much gentler speed (15 vs. 50) both still tripped *instantly* on the very first command at that setting – a pattern insensitive to speed is itself a clue against a straightforward "not enough dynamic torque" stall, since that failure mode should get *better*, not stay identically instant, as speed decreases.

The user's own comparison test settled it: after checking (this time correctly) that this NEMA17's rated current is 1.2 A – not 2.8 A, that number belongs to the unrelated NEMA23 originally installed on Joint X – the exact same command that had just tripped at 2000 mA and speed 15 was re-sent **without** `0x88` enabled. **It moved almost perfectly**: −1.0008° against a −1.0° commanded target (99.9%), with both `FD 01` and `FD 02` received. **This refutes "confirmed genuine stall" as this document previously claimed**: the identical command, current, and speed produced a clean, correct move the moment `0x88` was left off. `ENABLE_SHAFT_PROTECTION` on this driver appears capable of a false-positive trip – possibly reacting to a normal, brief startup transient present at the start of every move – not only a genuine sustained one.

Caution: `0x88` is off by default in this package's scripts – treat any trip as a lead, not proof

Both `joint_position_control_test.py` and `joint_stepped_move_test.py` now gate `ENABLE_SHAFT_PROTECTI` behind an explicit `--enable-shaft-protection` flag rather than sending it automatically, exactly because of the false-positive result above. If it does trip during real use, cross-check by re-running the identical command with it disabled before concluding a real stall occurred – and remember a trip latches the driver down and costs a power-cycle to clear, so triggering one by default on every session is expensive for a signal that isn't fully trustworthy yet.

2.12 Final resolution: a real, localized mechanical catch, found and fixed by hand

A proper statistical test settled the "incomplete move" question for good. `joint_b_statistical_test.py` (20 reps of ±0.5°, alternating direction so the joint returns near its start each pair, at the correctly-rated 1.2 A, speed 15, `0x88` off) came back **20/20 (100%)**, every rep within 1% of commanded magnitude, no degradation between the first and second half. The same script re-run for **sustained one-direction** motion instead of alternating reproduced the "burst then stop" pattern exactly: 3–4 good reps, then a hard wall – but critically, **starting a fresh sustained run already inside the previously-bad zone failed almost immediately** (1/10), while starting fresh from near 0° took several good reps first. That is the signature of a fixed *position*, not a command count, current level, or session-fatigue effect: alternating tests never accumulate enough one-directional travel to reach it, sustained tests do.

That localized it to roughly −2° to −5° in this joint's raw-command coordinate. Told to check that specific range slowly and deliberately (rather than a quick static check, which had found nothing earlier), the user felt **small catches while rotating through it by hand** – direct physical confirmation. Manually working the gears back and forth through the range (deliberately running the mechanism through the catch to seat/smooth it) was tried, retested, and iterated on the residual narrower sub-range that remained: an initial pass improved a −5° to −1° sustained test from 10% to 70% success; a second pass on the narrower −2° to −1.7° residual brought it to **100% (10/10) in both directions**, every rep 98–101% of commanded magnitude, confirmed across the full previously-affected range.

Caution: The whole "incomplete move" investigation was a real mechanical fault, not a CAN/firmware issue

Every earlier hypothesis this session – `0xF4` vs `0xF5` vs `0xFD`, working mode, current, re-enabling patterns, `FD 02` timing, shaft protection – was chasing symptoms of one real, physical, position-localized mechanical catch. `FD 02` presence remained a reasonably good proxy for "did this specific command finish" throughout (it degraded exactly where the catch was), but the actual fix was mechanical, found only by a slow, deliberate hand-check of the specific failing range once statistical testing had localized it. If a similar pattern appears on Joint C or any future joint – clean single moves, degrading specifically under sustained one-direction travel – localize the range the same way (alternating vs. sustained statistical tests) before assuming it is a settings or protocol problem.

2.13 Next steps

- Run a broader confirmatory sweep across more of Joint B's full calibrated range (`home_position` $+12.87°$ to `opposite_limit` $-24.23°$), not just the $\sim$5–10° window tested so far, to rule out any other undiscovered catch elsewhere in the range.
- Re-verify reliability at a more practical production speed (testing here used a gentle `speed=15`; confirm the fix holds at `speed=50` and above before relying on it for real trajectories).
- `ENABLE_SHAFT_PROTECTION`'s false-positive behavior (Section 2.11) is still unexplained and remains off by default – this session's resolution did not depend on it and does not change that finding.
- **Joint C**: try `SR_CLOSE` + 1600 mA + confirm CANRSP, matching Joint B's verified config. Phase-resistance check and motor/driver swap test are still pending from 2026-08-25. If similar incomplete-move symptoms appear, run the same alternating-vs-sustained statistical comparison early rather than assuming an electrical cause.

3 2026-08-27 (continued) — Large-scale motion, a second wall, and a gear-clearance hypothesis

With the $-2°$ to $-5°$ catch fixed and confirmed 100% reliable in both directions (Section 2.12), the user requested a much larger stress test: a full 360° rotation of the gearbox output, six times, in a bench configuration confirmed by the user to allow free/unlimited rotation (this specific setup does not have the joint's usual $\sim$37° calibrated envelope or any crossing cable at risk).

3.1 A real shaft-protection trip, and a lesson about tool-output reliability

The first full-360° attempt produced no visible movement, and its terminal output was lost to a tool error – a reminder that a lost or incomplete log means the actual hardware state must be re-verified from scratch (encoder position, enable state, protection state) before continuing, never assumed. Two subsequent attempts (a 30° command, then a 15° command) each tripped a **real** `ENABLE_SHAFT_PROTECTION` fault, visible directly on the driver's own physical display: `"Wrong Protect! Enter.."`. Unlike the earlier false-positive (Section 2.11), `0x88` appears to have stayed armed across the power-cycles performed earlier in the day (unlike `ENABLE_MOTOR`'s state, which

does not persist) and tripped for real partway into these longer moves. Pressing **Enter on the driver's own panel cleared the trip both times – no power-cycle was required**, updating the earlier assumption that a trip always needs a full power-cycle to clear. Explicitly sending `0x88 0x00` (disable) resolved the repeat trips.

3.2 The real behavior once protection was off: a stick-slip pattern, not a clean stall

With `ENABLE_SHAFT_PROTECTION` explicitly disabled, a 15° single command eventually completed (99.9% of commanded distance) but took a full **60 seconds** instead of the 1–2 seconds small moves take, moving in a distinct **stick-slip** pattern: long stalls of 10–30+ seconds at a fixed position, punctuated by sudden bursts (observed rates up to ∼5500 counts/s) that advanced several degrees at once, repeating 5–6 times before finishing.

3.3 Isolating the cause: removing the belt did not change anything

To test whether this was resistance in the belt-driven wrist mechanism downstream of the gearbox, the belt was physically removed and the identical 15° command re-run. **The stick-slip pattern was nearly identical** (same shape, similar total duration, if anything a longer single stall) with the belt off – despite the motor+gearbox now driving strictly less load than before. **This rules out the belt-driven wrist mechanism as the cause** and points at the motor/gearbox assembly itself.

3.4 Chunking into small commands did not avoid it either – it just found a different wall

Given every small isolated command earlier in the day was 100% reliable, the same 15° distance was tried as 15 separate 1° chunked commands (`joint_statistical_reliability_test.py`, sustained mode) rather than one large command. **Reps 1–6 were clean (99–101% each), then rep 7 was partial and reps 8–15 completely failed** (no `FD 02`, negligible motion) – a hard wall at a *different* absolute position (∼38–39° in this session's raw coordinate) than the originally-fixed −2° to −5° zone. Chunking did not solve the underlying problem; it simply relocated where the wall was encountered based on how far the sequence traveled before reaching it.

Caution: This looks like multiple distributed resistance points, not one isolated defect

Between the original catch, this new wall roughly 40° away, and a stick-slip pattern that persists across a 15° span independent of belt/load, the evidence now points at resistance distributed across the gearbox's own rotational range rather than a single fixable spot. The stick-slip signature (long stall, then sudden release) is also more consistent with tight/insufficient gear-tooth clearance (backlash) than with a single point of debris or damage – especially plausible for 3D-printed gears, where slight oversizing or ovality from the print process can cause mesh tightness to vary at different rotational positions.

3.5 Decision: reprint the gears with more clearance

Given the above, the user is decoupling the mechanism and reprinting the gears slightly smaller (more backlash) rather than continuing to locate and hand-work individual walls one at a time.

This is a well-motivated fix given the evidence – recommended refinements if pursued:

- Increase clearance modestly, not aggressively – too much backlash trades this problem for lost motion/imprecision in what is a precision joint.
- If the current gears show visible ovality or warping, consider print orientation/settings, not just uniform scale-down, since that would better explain why some rotational positions bind and others do not.
- Once reinstalled, re-verify across the **full range** using `joint_statistical_reliability_test.py` in both `alternating` and `sustained` modes, and at multiple starting positions – today's narrow-range tests gave false confidence more than once before a wider test surfaced the next issue.

3.6 Next steps

- Motor is disabled and the mechanism is being decoupled for the gear reprint. `can0` was brought down and `slcand` stopped per the standard shutdown procedure (Section C.2) before disconnecting the CANable.
- After reassembly with new gears, re-run the full bring-up procedure (Section E) from Step 1 – do not assume the CAN ID, calibration, or any prior finding still applies to the rebuilt mechanism.
- If the stick-slip pattern persists even after the gear reprint, that would argue against the backlash hypothesis and toward the motor/driver itself (bearing, encoder mounting, or a firmware behavior specific to long single commands) – keep the belt-off isolation result and the chunked-vs-single-command comparison as reference points for that follow-up.

4 2026-08-31 — Housing-off retest, gear reprint plan, and a definitive motor exoneration

4.1 Project now mirrored on GitHub

The custom tooling in this document (every script under `arctos_can_control`, the desktop control-panel app, CANable firmware images, and this SOP itself) is now published at `https://github.com/mdion2-maker/arctos_can_integration`. The upstream `ros2_arctos` repository is deliberately not duplicated there, since it already exists as its own project.

4.2 Housing-off retest: the stick-slip pattern got worse, not better

With the gearbox's outer housing removed (belt state at time of test not confirmed), the same 15° single-command test from Section 3 was repeated with the user watching the exposed mechanism directly. The stall was **longer than any prior test** – stuck at 25.7% of the commanded distance for a full **72 seconds** before suddenly bursting free and completing the remaining 75% in 4 seconds. The user again had to physically assist it to get moving – **confirming a real, sustained mechanical resistance, not a brief transient**. Removing the housing did not reveal an externally visible culprit, suggesting the resistance is internal to the gearbox (a specific stage, a bearing, or the shaft coupling) rather than something the housing alone exposes.

4.3 Decision: reprint the gears in Tough PLA with corrective settings

Given brass was less immediately available than nylon for the user, and given the diagnosis pointed at print-tolerance/backlash rather than a fundamental material limit, the decision was to reprint in Tough PLA (specifically Bambu Lab's own PLA Tough, printed on a Bambu Lab H2D) rather than jumping to metal. Settings recommended, layered on top of Bambu Studio's own built-in PLA Tough filament preset (which should supply temps/cooling – not overridden by generic figures):

- **XY Contour Compensation: -0.05 to -0.1mm** (Process Settings → Advanced) – shrinks the external tooth profile to loosen mesh tightness. This is the setting most directly targeted at the diagnosed problem; **XY Hole Compensation** (which affects internal cavities like the shaft bore, not the tooth profile) was left at 0 since the diagnosed resistance varies by rotational position – consistent with tooth-mesh tightness, not a bore/shaft fit issue, which would be relatively constant regardless of angle.
- Print orientation flat (gear axis vertical), not standing on edge – reduces ovality, which would otherwise reproduce the same position-dependent binding pattern in the new part.
- Layer height 0.12–0.16mm, 4–6 walls (or 80–100% infill given the small part size), slower outer-wall speed, moderate (50–70%) part-cooling fan.

4.4 Definitive result: the bare motor, fully decoupled, is completely clean

With the motor mechanically disconnected from the gearbox entirely (no belt, no gear coupling, no load whatsoever), `joint_statistical_reliability_test.py` was run twice using `--gear-ratio 1.0` (direct motor-shaft degrees, since no gearbox is attached to convert through):

- **Alternating**: 20 reps of $\pm 180°$ – **20/20 (100%)**, every rep within $0.3°$ of commanded.
- **Sustained**: 10 reps of $360°$ in the same direction ($3600°$ total, 10 full motor revolutions) – **10/10 (100%)**, every single revolution within $0.15°$ of exactly $360°$, with no stall, no hesitation, and no stick-slip pattern at any point across the entire sequence.

Caution: The motor, encoder, driver, and CAN chain are exonerated – the problem is confirmed 100% mechanical, inside the gearbox

This is the cleanest, most controlled test in the entire investigation, and it came back perfect at a scale (10 full revolutions, sustained, in one direction) that reliably triggered the stick-slip pattern in every prior test with the gearbox attached. Every earlier hypothesis about the motor, its encoder mounting, the driver's current/mode settings, or CAN-level command timing can now be set aside. The resistance is definitively located in the gearbox mechanism itself – fully consistent with, and now strongly supporting, the gear-reprint plan above as the correct fix rather than a guess.

4.5 Next steps

- Complete the gear reprint with the settings above; inspect the new gears for visible ovality/warping before installing (a quick visual/caliper check across a few diameters) rather than only discovering it via another multi-hour CAN test cycle.

- Reassemble, then re-run the full verification sequence from Section 2.12 onward: `can_id_scan.py` (do not assume the ID survived reassembly), `joint_reliability_test.py`, then `joint_statistical_reliab` in both modes across as much of the range as practical, motor-only baseline (Section 4.4) already established as the reference for "clean."
- If the sustained-mode test still shows any stick-slip after reassembly with the new gears, that would point at a bearing or shaft-alignment issue distinct from tooth-mesh clearance, since the mesh-tightness hypothesis would have been directly addressed by the reprint.

5 Joint C — SR_CLOSE applied, then the same motor-exoneration pattern as Joint B

5.1 SR_CLOSE + 1600 mA + confirmed CANRSP: still no torque connected

Applying Joint B's verified-working panel configuration to Joint C (Section 2) – `SR_CLOSE`, 1600 mA, CANRSP enabled – did **not** produce holding torque while connected to the gearbox; the shaft still span freely. Joint C has now been tried in every documented working mode (`SR_vFOC`, `CR_vFOC`/`CR_FOC`, `CR_CLOSE`, `SR_CLOSE`) across 1200–3000 mA, with zero holding torque in any combination while connected – ruling out a simple settings mismatch as the sole explanation.

5.2 A confirmed real move – then inconsistent torque

Watching the panel for a fault message (the technique that found Joint B's shaft-protection trip) revealed no error, but did surface something new: after unplugging and replugging the motor connector, a commanded 0.5° move executed almost perfectly (0.4963° actual, confirmed independently by both the CAN-side encoder and the driver's own panel display, which reports motor-shaft-scale degrees rather than joint-scale). **This is Joint C's first confirmed real movement across the entire investigation.** Immediately afterward, though, holding-torque checks were inconsistent – spinning freely both right after the move and during two subsequent 10s/20s enable-and-hold tests (the second including deliberately wiggling the connector, which changed nothing) – while the connector was still attached to the gearbox.

5.3 Bare-motor test: fully exonerated, exactly like Joint B

With the motor mechanically disconnected from the gearbox (mirroring Section 4.4's test on Joint B), using `joint_statistical_reliability_test.py` with `--gear-ratio 1.0`:

- **Alternating**: 20 reps of ±180° – **20/20 (100%)**, all within 0.15° of commanded.
- **Sustained**: 10 reps of 360° (3600° total, 10 full revolutions) – **10/10 (100%)**, essentially exact.
- **Static holding torque, isolated**: enabled and held for 10s with no move command – **the shaft resisted turning by hand**, confirmed real holding torque, with the motor noticeably warm afterward (consistent with genuine current draw).

Caution: Joint C's motor and driver are healthy – every failure observed was downstream of the connector, in the gearbox

This flips the earlier "loose connector" theory: a genuinely flaky connector should have caused failures during this 30-command bare-motor test too, and it did not – perfect reliability throughout, for both motion and static holding torque. The far more consistent explanation across every observation this session: **the motor and driver are completely fine**, and the one successful move plus the otherwise-consistent "spins freely" results were the gearbox itself intermittently binding – occasionally overcome by an active move command, but resisting during passive holding. This is the same root-cause category as Joint B (Section 4.4), on the same 67.82:1 gearbox family.

5.4 Next steps

- Let the motor cool (it was warm after the holding-torque confirmation) before further enable tests.
- Reconnect Joint C's gearbox and repeat the alternating-vs-sustained comparison used to diagnose Joint B – if the same stick-slip pattern appears, treat it with the same hands-on approach (localize by feel, work the gears, or reprint with the same corrective settings from Section 4).
- Given both Joint B and Joint C – built from the same driver/gearbox family – have now independently converged on "motor and driver clean, gearbox at fault," consider whether this points at a systematic issue in how this gearbox family is manufactured/printed rather than two unrelated one-off defects.

6 Joint A — CAN confirmed, bare-motor exoneration, and two script bugs found

6.1 Hardware identity: matches the original Joint X spec, but this driver is the CAN variant

A newly-connected joint, described as NEMA23, 1.8 Nm, 2.8 A, was labeled "Joint A" by the user – the exact motor spec originally recorded for "Joint X" at the very start of this project (Section 1, the joint whose driver turned out to be an RS485 variant, not CAN). Before assuming this was the same failed unit, the bus was scanned directly rather than trusting either the driver panel or `arctos_controller.yaml`'s `A_joint` entry (which is a `MKS_42D`, NEMA17-class config and does not match this motor's spec at all). The scan found exactly one responder, at CAN ID `0x01`, with a clean checksum-verified reply – matching `X_joint` in the real config (`motor_id: 1`, `gear_ratio: 13.5`, `hardware_type: MKS_57D`), not `A_joint`. This confirms two things at once: the naming the user is using for this joint does not match the letter-keyed config, and – more importantly – **this is a genuine CAN-responding driver**, not a repeat of the original RS485 hardware mismatch.

6.2 Bare-motor verification, decoupled from any belt/drivetrain

With nothing attached (the motor was not connected to any belt/pulley at the time of testing), the full diagnostic ladder used previously on Joints B and C was repeated:

- **Link quality** (`joint_reliability_test.py`): 200/200 responses (100%), 0 CRC failures, 0 encoder jitter while stationary, mean round-trip latency 3.3 ms.

- **First motion** (`joint_micro_move_test.py`): commanded 300 raw counts (≈ 0.488°), observed 301 raw counts (≈ 0.490°) – essentially exact, no slip.
- **Monitored 3-step move** (`joint_stepped_move_test.py`): three 2° relative steps, each landing within about 1% of commanded, no missed steps.
- **Alternating statistical test** (`joint_statistical_reliability_test.py`, `--gear-ratio 1.0`): 30 reps of 3° – **30/30 (100%)**, both halves 100%.
- **Sustained statistical test**, run faster and with a bigger step per the user's request (speed 40 instead of the usual gentle 15, inter-rep pause cut from 2.0 s to 0.5 s, 25° per rep): **10/10 (100%)**, both halves 100%, total travel 248.9°.

Same clean profile as both prior bare-motor exonerations (Joint B, Section 4.4; Joint C, Section 5): this motor, driver, encoder, and CAN link are all healthy.

Caution: Two real bugs found in the shared test scripts, not hardware faults – both fixed

Two issues surfaced while testing Joint A that would affect any joint with the same characteristics, and have already been corrected in `joint_stepped_move_test.py`, `joint_statistical_reliability_test.py` `joint_long_move_monitor.py`, `joint_position_control_test.py`, and `joint_streamed_move_test.py`:

1. **Safe-band ordering assumed `home_position > opposite_limit`.** The band was computed as `lo = opposite_limit + margin`, `hi = home_position - margin`. This only forms a valid ascending range when `home_position > opposite_limit`, which happens to be true for Joints B and C but is *false* for Joint A/X (`home_position` = −173.5°, `opposite_limit` = +157.5°). With the old code this made `lo > hi`, so *every* move was rejected as leaving the safe band regardless of direction. Fixed by using `lo = min(home, opposite) + margin`, `hi = max(home, opposite) - margin`, which is correct for either convention.
2. **The safe-band projection and the `pct` reliability metric assumed commanded sign equals physical sign.** For any joint with `inverted: true` in `arctos_controller.yaml` (true for every joint in this project: X, Y, A, B, C), the encoder consistently moves opposite to the commanded direction byte. The pre-move safe-band check projected the *commanded* sign forward, so it could both block a command that would have physically moved back into the safe band, and – more seriously – fail to catch a command that was nominally moving away from a limit but physically moving toward one. It also made every printed `pct` value read as roughly −100% instead of +100% on perfectly good moves (visible throughout the alternating-test output in this session), which could be mistaken for a systematic problem. Fixed by adding an opt-in `--inverted` flag to `joint_statistical_reliability_test.py` that flips the sign used only for the projection and the `pct` comparison, leaving the actual command byte sent to the driver unchanged.

Neither issue reflects a hardware problem; both were caught because Joint A's config has a different sign convention (`home_position` < `opposite_limit`) than Joints B and C, which is exactly the kind of case that ought to be checked before trusting a script that was only validated against one convention.

6.3 Next steps

- Reconnect Joint A to its belt/drivetrain and repeat the alternating-vs-sustained comparison at realistic (coupled) load, watching specifically for the same stick-slip signature found on Joints B and C.
- Pass `--inverted` on every future `joint_statistical_reliability_test.py` run for a joint marked `inverted: true` in `arctos_controller.yaml` – i.e. essentially always, in this project.
- Resolve the naming mismatch between the user's "Joint A/B/C/X" labels and the config's letter-keyed `A_joint`–`C_joint`/`X_joint` entries before it causes a real mix-up; CAN ID `0x01` is this joint's actual identity regardless of which label is used for it.

7 2026-08-31 (continued) — Joint A belt-coupled verification

With the belt reconnected, the same three-step ladder used on the bare motor (Section 6) was repeated at real load, using the actual `gear_ratio: 13.5`:

- **Supervised first motion** (`joint_micro_move_test.py`): commanded 300 raw counts, observed exactly 300 – no slip.
- **Alternating statistical test** (30 reps, 3° steps, `--inverted`): **30/30 (100%)**, both halves 100%, every rep within about 1% of commanded, net drift essentially zero.
- **Sustained statistical test** (10 reps, 5° steps): **10/10 (100%)**, both halves 100%, 99–101% accuracy on every rep across the full ∼50° traveled.

No stick-slip signature at all under real belt load – Joint A is the first joint in this project to pass both the bare-motor and belt-coupled comparisons cleanly on the first attempt. Per the standing project plan, Joint B testing resumes once confidence in Joint A is established; that is what the next session log covers.

8 Joint B — interrupted calibration, a runaway-looking symptom, and recovery

---

Caution: This section documents a real safety incident, not just a diagnostic result

The motor was disconnected from its gearbox at the time (bare shaft, no load, no obstruction), which is what kept this from being worse than it looked. Read this before changing a working mode on any driver in this project.

8.1 What happened

While reconnecting to Joint B, the user changed the driver's working mode to `SR_CLOSE` on the panel. The motor then began spinning fast and continuously and did not stop on its own. Unplugging the CANable adapter (removing the CAN link) did *not* stop it – the user had to cut power to the motor/driver directly. On a second attempt to resolve it (letting the panel's own routine run

again), the same thing happened, this time also accompanied by an audible squeak, prompting an immediate power cut again. At each point the correct action was taken: cut motor power first, ask questions about root cause second.

8.2 Diagnosis

Two details narrowed this down:

- The motor was confirmed bare (no gearbox attached) and unobstructed both times, which ruled out the squeak being a load-bearing mechanical strain (e.g. a gear grinding) – consistent instead with normal stepper coil whine at the speed the routine was running at.
- The panel's position LED tracked rotation normally while it was happening, meaning the encoder itself was being read correctly – this argued against a corrupted/garbage encoder value driving a genuine closed-loop runaway, and for a simpler explanation: **changing the working mode to `SR_CLOSE` auto-triggers an encoder self-calibration routine on this driver, and that routine was interrupted (twice) before it could finish.** Some MKS firmware re-attempts an incomplete calibration on every subsequent power-up, which matches the user's report that it happened again, immediately, the next time it was powered on.

8.3 Recovery

Since the motor was bare and unobstructed, the risk of letting the routine run was mechanical-shaft-safety only (keep hands, hair, and loose clothing clear of a fast-spinning exposed shaft), not load/damage risk. The recommendation was to power on, touch nothing, and time it rather than interrupt again. This worked: the calibration completed on its own, the working mode was kept at `SR_CLOSE`, and the user also changed the driver's CAN ID to `0x02` on the panel at the same time.

Caution: Changing a driver's working mode can trigger a real, uninterruptible-looking calibration – let it finish

If a driver starts moving on its own immediately after a working-mode change and won't stop, the first move is always to cut power to the motor/driver itself – unplugging only the CAN adapter does not stop a firmware-internal routine like this. But if the motor is confirmed bare/unobstructed, letting a suspected calibration routine run to completion (rather than interrupting it again) is usually the faster path back to a working state, since an interrupted calibration can leave the driver re-attempting it on every power-up rather than settling.

8.4 Post-recalibration verification

The new CAN ID was verified by scan, not assumed from the panel (per the project's standing rule): `can_id_scan.py` found exactly one responder, at `0x02`, confirming the panel's reported value was correct this time. With the motor still bare (`--gear-ratio 1.0`):

- **Link quality**: 200/200 responses (100%), 0 CRC failures, 0 encoder jitter, mean latency 2.7 ms.
- **First motion**: commanded 300 raw counts, observed 304 – within about 1.3%, no slip.

An alternating-vs-sustained statistical comparison followed, bare motor, `--gear-ratio 1.0`, using generous ±360° testing bounds rather than the real narrow joint-level limits (which do not apply while decoupled):

- **Alternating** (30 reps, 3° steps): every single rep landed within 0–2% of commanded magnitude, with no degradation across the run – but **0/30 reps ever reported the `FD 02` status byte** this script treats as its success signal; every rep instead only ever showed status `1`.
- **Sustained** (10 reps, 20° steps): the same pattern held – 99–101% accuracy on every rep, 200.3° traveled with zero degradation, and again **0/10 reps reported `FD 02`**. Consistent across both modes, this looks like a stable behavioral change rather than a one-off fluke.

Caution: Perfect motion, but the `FD 02` success status never appeared – don't misread this as 0% reliability

This is not a repeat of the earlier false-positive lesson from `ENABLE_SHAFT_PROTECTION`, but it rhymes with it: the SOP already documents (Section 2, "Characterizing `0xFD`") that this status byte is a real signal with a still-unknown trigger condition, not a required precondition for real motion. Here, every rep's actual encoder-measured movement matched the commanded value almost exactly, alternating direction cleanly 30 times in a row with zero drift – physically, this is a perfect result. The status frames simply never included the specific byte this script's success check looks for, only a different one (`1`) that Joint A's equivalent tests always paired with `2`. Whether this is a benign side effect of the just-completed recalibration and CAN ID change, or a genuine behavioral difference worth tracking, is not yet known – but the fix is not to distrust the motion, it is to stop treating `FD 02` as the only valid definition of success in this script.

9 A real runaway, traced to an absolute-position encoding bug, not the gearbox

Caution: A confirmed software bug, not a mechanical or calibration fault – but it produced a genuinely dangerous incident

With Joint B's reprinted gearbox reconnected, a routine supervised first-motion test (`joint_micro_move_test.p` commanding a tiny ∼ 0.1° nudge) instead produced a large, continuous, non-settling motion that did not stop within the test's 5-second watch window. The user had to cut motor power directly. This is documented in full because the same class of bug is present in three other scripts and needs to be understood, not just patched around.

9.1 What the test actually showed

The encoder trace during the 5-second window was smooth and strictly monotonic (roughly −2080 raw counts per 0.5 s sample, ten samples straight, no deceleration) – not noise, not a stall, not a protocol error. It looked exactly like a real, large, deliberately-commanded move that simply hadn't finished yet. The command box itself confirmed this: commanded 300 raw counts (≈ 0.097°), observed −20,871 raw counts (≈ −6.76°) in just those five seconds, with no sign of nearing a target.

9.2 Root cause: absolute-position encoding breaks once the raw counter has drifted far from zero

`joint_micro_move_test.py` computed an absolute target (`target_raw = raw0 + delta_counts`) and packed it into the driver's 3-byte position field with a plain `int(target_raw) & 0xFFFFFF` – ordinary two's-complement truncation. This has been in the script since it was written and never caused a problem before, because every prior test on every joint started from a raw position near zero (a few hundred counts at most), where two's-complement truncation of a small number is indistinguishable from the driver's real encoding.

Joint B's raw absolute counter, however, had drifted to roughly −274,000 counts by this point in the session – the accumulated result of the earlier bare-motor alternating and sustained tests (Section 8), none of which ever re-homed or zeroed the joint, because none of them needed to: they all use *relative* moves, so absolute drift never mattered to them. But `joint_micro_move_test.py`'s absolute-target math did depend on it. Packing −273,639 (start −273,939 plus the 300-count nudge) via plain 24-bit two's complement produces the byte pattern `FB D3 19`. Decoded the way this driver family actually reads a position field – a direction bit in the top bit of the first byte, and a 23-bit *unsigned magnitude* in the rest, not full two's-complement – that same bit pattern means direction bit set, magnitude $0{,}x7BD319 = 8{,}111{,}385$ pulses, i.e. roughly 178,220°. A 300-count nudge was, by encoding accident, transmitted as a request to travel almost 500 revolutions. That fully explains a large, smooth, non-settling motion with no error reported – the driver was doing exactly what it was (wrongly) told.

9.3 Fix applied

`joint_micro_move_test.py` was switched from `ABSOLUTE_POSITION (0xF5)` with a raw absolute-target packing to `POSITION_CONTROL (0xFD)` with a direction byte plus an always-*positive* magnitude – the same pattern already proven safe across every joint in this project via `joint_statistical_reliability_` and `joint_position_control_test.py` (thousands of commands, zero encoding failures, across Joints A, B, and C). This pattern never references the joint's current absolute position at all, so it cannot repeat this failure regardless of how far a joint's raw counter has drifted.

Caution: The first version of this fix was still off by 5.12x – a units mismatch, not a repeat of the encoding bug

A follow-up guarded re-test (small move, active 1° safety cutoff polled continuously rather than just watched) landed at 0.4976° against a commanded 0.0972° – a real, repeatable 5.12× overshoot, though bounded and settled, not a runaway. Cause: `POSITION_CONTROL`'s pulses field is in *motor microstep* scale (`MICROSTEPS_PER_MOTOR_REV = 3200`, i.e. 200 full steps × 16 microsteps), not the 16384-count encoder scale `joint_micro_move_test.py`'s `--delta-counts` was already defined in. The first fix passed the encoder-scale number straight through as pulses; 16384/3200 = 5.12, matching the overshoot exactly. Corrected by converting: `pulses = round(delta_counts * MICROSTEPS_PER_MOTOR_REV / ENCODER_CPR)`, matching the conversion already used correctly in `joint_statistical_reliability_test.py`. Re-tested clean: 0.0969° actual against 0.0972° commanded, settled immediately, no cutoff needed.

Caution: Three other scripts share the same class of risk and have not yet been fixed

A repo-wide check found the identical unguarded `& 0xFFFFFF` packing in:

- `joint_stepped_move_test.py`, `--mode absolute` – packs a full absolute `target_raw`, the exact same pattern that just failed. Unverified and should be treated as unsafe on any joint with a large or unknown drifted position until fixed.
- `joint_stepped_move_test.py`, `--mode relative`, and `joint_streamed_move_test.py`'s `move_relative()` – pack a possibly-*negative* relative delta the same unguarded way, rather than splitting sign into a direction byte and magnitude. Whether a small negative delta is actually misread the same way has not been empirically confirmed either way; treat as unverified rather than assumed-safe.
- `mks_driver.py` – has the same absolute-target packing pattern (`move_relative_degrees`), guarded only by a hardcoded ±71,500-pulse boundary check with no stated derivation. This file is already documented elsewhere in this SOP as containing placeholder/untrustworthy calibration numbers copied from an unrelated joint; it also has an outright bug independent of this one (`__init__` references `self.PULSES_PER_MOTOR_DEGREE`, which is never defined, so instantiating the class would raise `AttributeError`). Do not treat its ±71,500 guard as a validated safe boundary – it has not been re-derived or confirmed here.

Until these are fixed and verified, prefer `joint_statistical_reliability_test.py` / `joint_position_control_te` for any real motion test – they are the only scripts in this project with an extensive, confirmed-safe track record for exactly this reason.

9.4 A note on the earlier Joint B "partial burst then stop" investigation

`joint_stepped_move_test.py`'s own header notes it was built while chasing Joint B's original 2026-08-27 "commands get acknowledged but only produce a partial burst of motion" behavior, using `--mode absolute` test runs among others. That investigation's eventual conclusion – a real, physical gear-mesh clearance problem, confirmed by hand and by the proven-safe 0xFD statistical tests – is not in doubt; it was independently corroborated multiple ways, including bare-motor exoneration. But any individual `--mode absolute` result from that session should now be read with this encoding risk in mind, in case it added noise on top of the real mechanical signal.

9.5 Next steps

- Fix `joint_stepped_move_test.py` and `joint_streamed_move_test.py` to use the same direction-byte-plus-magnitude `POSITION_CONTROL` pattern, then re-verify each against a joint with a deliberately large drifted position before trusting them again.
- Do not use `mks_driver.py` as-is; it needs the same fix plus resolution of the undefined-attribute bug before it can be trusted for anything.
- Consider adding a periodic homing/re-zero step to the bare-motor statistical tests specifically to keep raw counters from drifting this far in the first place, independent of fixing the encoding bug itself.

10 Joint B real-load verification, after re-establishing a safe envelope by hand

10.1 The calibrated home/opposite-limit values were no longer trustworthy

Before testing at real gearbox load, a fresh position read showed the joint at −175.14° by the software's math – far outside `arctos_controller.yaml`'s real calibrated range for this joint (−24.23° to 12.87°, about 37° wide). This is a direct consequence of every bare-motor test in this session (Sections 8, 9) being relative-move-only and never re-homing: hundreds of degrees of untracked rotation accumulated, so the raw counter is no longer anchored to the joint's real physical zero. With the gearbox now attached and a real, narrow mechanical range, trusting the stale calibrated bounds against this drifted reading could have driven the joint into a hard stop with no warning, since the safe-band check only protects against what it's told, not against a wrong reference frame.

10.2 Re-establishing a safe envelope by hand instead of trusting stale calibration

Rather than guess, the joint's physical position was checked by hand: with coils disabled, the user reported it currently sits pointing straight up, with roughly 30° of free travel available in each direction before a hard stop. That number, not the stale calibrated values, was used to build the test's safe band: a fresh raw read gave −175.144°, and `--home-position-deg -205.14 --opposite-limit-deg -145.14` (current position ±30°) was passed to `joint_statistical_reliability_test.p` giving roughly ±27° of usable room after the script's default 3° margin.

10.3 Results: clean at real load, matching the bare-motor result exactly

- **Alternating** (20 reps, 2° steps): **100% magnitude accuracy on all 20 reps**, net drift +0.0023° over the whole run – effectively zero.
- **Sustained** (5 reps, 5° steps, chosen to stay within the ±27° confirmed-safe room): **100% accuracy on all 5 reps** (99.97–100% each), 25.0° traveled essentially exactly as commanded, ending at −200.14°, comfortably inside the safe envelope.

As with every test since the recalibration (Section 8), `FD 02` never appeared in either run – consistent with the already-documented behavioral change, not a new concern. No stick-slip signature at all under real load. Joint B now matches Joint A: clean on both the bare-motor and real-load comparisons, on the reprinted gearbox.

Caution: Re-home Joint B properly before trusting absolute-degree commands again

This session's safe envelope was built from a hand-estimated ±30°, which was good enough for a supervised statistical test but is not a substitute for real homing. Before running anything that trusts `arctos_controller.yaml`'s `home_position`/`opposite_limit` values again (or before any unsupervised/ROS-driven motion), re-home Joint B properly so its raw counter is re-anchored to a known physical reference.

Caution: `ENABLE_MOTOR 0x00` does not appear to interrupt a `POSITION_CONTROL` move already in progress

Returning Joint B to its confirmed "straight up" reference position (−175.14°) after the tests above, a guarded single relative move undershot its 8-second monitoring window and the script disabled the motor (`0xF3 0x00`) and exited while still 14° short. Re-running the same script moments later found the joint had continued on toward the target well past where monitoring stopped – the move kept executing on the driver's own internal control loop after the disable command was sent, not just after the script exited. **Disabling coils does not appear to be an immediate stop for a move already underway; `EMERGENCY_STOP (0xF7)` is the actual immediate-halt command.** This session it resolved safely only because the residual motion happened to continue in the correct direction and stayed inside the confirmed-safe envelope the whole time. Any future guarded test that might need to actually interrupt an in-flight `POSITION_CONTROL` move should send `0xF7`, not rely on disabling coils.

Caution: Correction: everything labeled "Joint B" in this session (Sections 8–10) was actually Y_joint, not B_joint

After this session's testing was done, the user realized the naming had drifted: the physical unit being called "Joint B" throughout the recalibration incident, the absolute-position encoding bug, and the real-load verification was actually `Y_joint` (`motor_id: 2, gear_ratio: 150.0, hardware_type: MKS_57D`) – confirmed by the fact that the CAN ID found during that session, `0x02`, is exactly Y_joint's real `motor_id`, not B_joint's (`5`). Separately, "Joint A" throughout this document is confirmed correct as `X_joint` (already used with the right `gear_ratio: 13.5` throughout).

The real B_joint (`motor_id: 5, gear_ratio: 67.82, MKS_42D`) is the joint referred to earlier in this document under the original Joint B gear-backlash investigation (Sections 2–4.4), from before "Joint A" existed in this project's vocabulary – that work is unaffected by this correction.

Practical impact of this session's mislabeling: every script run against the mislabeled joint used `--gear-ratio 67.82` (B_joint's value) instead of the correct `150.0` (Y_joint's value). Since `joint_deg = raw * 360 / 16384 / gear_ratio`, using the smaller gear ratio made every reported "joint degree" figure read about $150/67.82 \approx 2.21\times$ *larger* than the true physical rotation – the real motion was smaller than reported, not larger. The hand-measured ±30° safe envelope (Section 10) was likewise computed in the wrong-scaled unit, which made the enforced raw-count safety bound *tighter* than the true physical limit, not looser. No part of this correction indicates an actual safety violation; it indicates the reported degree figures for that joint in this session should not be trusted at face value, and any future work on this specific unit should use `gear_ratio: 150.0` and Y_joint's real calibrated `home_position`/`opposite_limit` (`-0.854`/`2.006` rad) going forward.

---

11 Joint Z bring-up (bare motor)

11.1 Settings and calibration

Panel configured to `SR_CLOSE`, 1600 mA (matching the working baseline established for the other MKS_42D-family joints), and calibrated – this time letting the routine run all the way to completion and settle on its own, per the lesson learned from Y_joint's interrupted-calibration incident (Section 8). CAN ID verified by scan (not assumed from the panel) as `0x03`, matching `Z_joint`'s real `motor_id` in `arctos_controller.yaml` exactly – no naming mismatch this time.

Caution: Initial reliability/first-motion results were reported at the wrong scale, then corrected

Before confirming the motor was bare (decoupled from its external gearbox), the reliability and first-motion tests were run with `--gear-ratio 150.0` out of habit. Once confirmed bare, this was corrected to `--gear-ratio 1.0` for the remaining tests, matching this project's standing convention for every other bare-motor test. The first-motion result was re-derived: the commanded 300 raw-count move was actually ≈ 6.50° of real motor-shaft rotation (observed ≈ -296 counts ≈ −6.50°), not the ±0.04° originally reported at the wrong scale. As with the Y_joint mislabeling above, the raw CAN commands and actual physical motion were correct throughout – only the degree-unit reporting was off, caught before any statistical testing was run at the wrong scale.

11.2 Results: clean on the bare motor

- **Link quality**: 200/200 responses (100%), 0 CRC failures, 0 encoder jitter, mean latency 3.4 ms.
- **First motion** (corrected scale): ≈ 6.50° commanded and observed, matching closely.
- **Alternating** (30 reps, 3° steps, `--gear-ratio 1.0`, `--inverted`): **30/30 (100%)**, both halves 100%, consistent ∼97% magnitude accuracy, `FD 02` present on every rep (matching Joint A/X's normal behavior, unlike Y_joint's post-recalibration change).
- **Sustained** (10 reps, 20° steps): **10/10 (100%)**, both halves 100%, 98–102% accuracy, 200.4° traveled with no degradation.

Joint Z is clean on the bare motor, matching the pattern for every other joint verified so far in this project. Not yet tested at real load (still decoupled from its external gearbox/drivetrain at time of writing).

12 Joint Z real-load testing — a chain of apparent mechanical problems, all traced to one test-script bug

12.1 Safety envelope: a correction mid-session, then a real hard-stop scare avoided

With the belt attached, the user initially reported ±20° of room by hand, then corrected this to ±10° before any test that mattered was run at the wider figure. The correction mattered: the first alternating test at very slow speed (5) drifted a net −10° over 10 reps and initially looked like it might explain a suspected gear-backlash problem – but re-examining it against the corrected ±10° figure, the final position landed almost exactly on the true boundary, raising the possibility the joint had simply driven into a real hard stop rather than exhibited a mechanical fault. The user confirmed nothing was jammed and manually recentered the joint, reporting the same ±10° figure from the new middle.

12.2 The reversal problem was real, not a hard-stop artifact

Retesting at the same slow speed from a freshly-centered, properly-verified-safe position reproduced the identical failure pattern almost exactly (`dir=+` moving $\approx -0.81°$ against a $-2.0°$ command; `dir=-` moving $\approx -1.19°$ against a $+2.0°$ command – wrong sign, not just wrong magnitude), aborting safely via the safe-band check partway through. Since this was nowhere near a hard stop this time, the hard-stop explanation was ruled out: this is a real, repeatable behavior specific to alternating direction under load at very slow speed.

12.3 A clarifying single-direction test, then full resolution at normal speed

A single sustained-direction corrective move (recentering the joint, still at speed 5) progressed smoothly and accurately toward its target with no sign of the reversal problem – confirming the issue is specific to *alternating* direction, not slow speed in general. Retesting the alternating comparison at normal speed (15) from the same properly-centered, verified-safe position came back completely clean: **10/10 (100%)**, every rep within 0.3% of commanded magnitude, net drift $+0.0001°$.

Caution: Working theory at the time: backlash needing momentum to close on reversal – see full retraction below

This was the working explanation reached in-session for the slow-speed alternating result above. It turned out to be the wrong explanation for a related-looking but distinct set of symptoms found next (the sustained-test "stiction" pattern), which was fully retracted after finding a real script bug. Whether *this specific* slow-speed alternating finding is itself an artifact of the same bug, rather than a real backlash/momentum effect, is addressed at the end of this section – read to the end before trusting either conclusion.

12.4 A "stiction" pattern that was actually a test-script bug – full retraction

The sustained tests that followed (documented in earlier drafts of this section) appeared to show real partial-completion behavior at 3° and 5° steps – undershooting on a move's first attempt, sometimes recovering on a second attempt, sometimes not – and were provisionally attributed to mechanical stiction (static friction higher than kinetic, needing a "warm-up" move to fully engage). **This conclusion was wrong, and has been fully retracted.**

The actual cause: `joint_statistical_reliability_test.py` read the encoder exactly once, a fixed `--status-timeout-s` (default 4.0 s) after sending each move, regardless of whether the move had actually finished. On Joint Z's gear ratio (150, much larger than Joint B's 67.82 this script was originally tuned against), moves of a few degrees at normal speed genuinely take 5–8+ seconds to complete. Every rep was therefore measured mid-motion. The shortfall then finished silently during the 2-second inter-rep pause, uncounted, and got folded into the *next* rep's reported delta – fabricating a completely artificial "first move undershoots, later moves recover" pattern that had nothing to do with the joint's real mechanical behavior.

This was confirmed conclusively by watching two isolated single moves (a 5° and a 3°) continuously for up to 20 seconds with no fixed cutoff: both reached their commanded distance almost exactly ($-5.0010°$ and $+3.0022°$ respectively) and held rock-steady afterward – they were never stuck or resisting, they simply took longer than the old test script waited.

Fix applied: `joint_statistical_reliability_test.py` now polls the encoder until the position genuinely stops changing (three consecutive stable readings) instead of reading once on a fixed

clock, with `--status-timeout-s` repurposed as a generous upper bound (default raised to 15 s) rather than a fixed wait. Re-running both step sizes with the fixed script came back completely clean: 3° sustained, 5 reps, **100% accuracy on every single rep including the first** (−3.0045° to −2.9962°), with `FD 02` present on every rep this time; 5° sustained, 6 reps, **100% accuracy on every rep** (+4.9970° to +5.0029°), `FD 02` present throughout. There is no stiction, no step-size dependency, and no completion problem at normal speed on this joint – the joint has been behaving correctly the entire time.

Caution: Re-verified: the slow-speed "reversal problem" was the same bug too – fully retracted

Re-run with the fixed script (10 reps, 2°, alternating, speed 5): **10/10 (100%) magnitude accuracy in both directions**, `FD 02` present on 9 of 10 reps. No wrong-sign moves, no partial completion, no reversal problem of any kind. The entire "low-speed reversal failure" finding earlier in this section was the identical fixed-timeout measurement bug, not a real behavior – there is nothing wrong with Joint Z at slow speed either.

12.5 Conclusion: Joint Z has no mechanical issues at any speed or step size tested

Every real-load finding in this section that suggested a mechanical problem – the low-speed reversal failure, the step-size-dependent under-completion, the "stiction" recovery pattern – was the same single root cause: the test script measuring position before slower moves had actually finished. With that fixed, Joint Z is clean across bare-motor and real-load testing, both directions, multiple step sizes, and both slow and normal speed. This is a good example of why an odd result should prompt checking the measurement method before concluding the hardware is at fault.

12.6 Next steps

- Apply the same settle-detection scrutiny to any other joint's earlier results that used the old fixed-timeout version of this script, particularly any with a large `gear_ratio` where moves take longer to complete (Y_joint at 150) – Joints A/X (13.5) and B/C (67.82) have smaller gear ratios and shorter move durations, making this less likely to have affected them, but it has not been explicitly re-checked.
- Sync the fixed `joint_statistical_reliability_test.py` to the public repo and note the fix in its own history, not just here.

13 2026-09-03 — Joint C: endstops made readable, and verified motion

Two things that had been blocked for several sessions were resolved: the endstop sensors became visible to the driver, and the joint was confirmed to drive its load accurately. Neither turned out to be the hardware fault previously recorded.

13.1 The endstop fault was configuration, not wiring or sensors

Across two sessions and **two different sensors**, a hall endstop on Joint C would light its own LED under a magnet while `READ_IO (0x34)` never changed. The 2026-08-25 line of reasoning concluded the break lay between the sensor's output pin and the driver's input pin. It did not.

The Arctos build does not use the vendor's default endstop wiring. The MKS manual's default puts a limit switch on the `IN_1` terminal of the CAN-side connector, but the Arctos documentation instructs builders on 42D axes to enable **limit port remap**, which moves the left limit onto the `En` pin and the right limit onto `Dir`, and requires `Com` to be tied to the matching high level. This arm is wired that way: the sensor's north output on `En`, its south output on `Dir`.

The remap is not enabled by default, and it had never been sent to this driver. Without it, bit 0 of `0x34` reports the physical `IN_1` terminal — which is correctly empty on this build — so it idled high forever no matter what the magnet did. The driver was faithfully reporting an unconnected pin. Enabling the remap changed the resting byte from `0x01` to `0x03` instantly, because bit 1 stopped being a nonexistent 42D input and started reporting the `Dir` pin.

```
# 0x9E enable=01 -- CAN ID 0x06, checksum 0x06+0x9E+0x01 = 0xA5
cansend can0 006#9E01A5
# reply 9E 01 A5  -> status = 1, set success
```

Listing 1: Enable limit port remap on Joint C

Caution: There is no read-back for the remap flag

The manual documents `0x9E` as a write only. Nothing reports whether remap is currently on, so it cannot be queried — only set. If a future session finds an endstop silent again, re-send `9E 01` before suspecting wiring.

13.2 Verified bit mapping and trigger polarity

With remap enabled, and measured against both magnet poles:

Bit	Reports	Pin	Triggered by
bit 0	`IN_1`	`En`	north pole
bit 1	`IN_2`	`Dir`	south pole

Both idle **high** and read **low** when triggered, so the default `Hm_Trig = Low` is already correct and needs no change. Resting byte is `0x03`; north alone gives `0x02`, south alone `0x01`. The two never went low together — no cross-talk between the outputs.

13.3 Prerequisites that must be right for any of this to work

- **`Com` tied to 5 V.** `En` and `Dir` are opto-isolated; with `Com` floating they cannot register anything. The manual is explicit: if the `STP/DIR/EN` signal high level is 5.0 V, `Com` must be 5.0 V.
- **A serial work mode.** The manual restricts remap to serial control mode, and in any `CR_` mode `En` and `Dir` *are* the pulse-interface enable and direction inputs. This joint was set to `SR_vFOC`.

- **No DIP switch is involved on the 42D.** The `SW1` pin-2 instruction that circulates for this board belongs to the 57D half of the manual. On the 42D, `EVCC/EGND` is the board's own 5 V, rated 20 mA.
- `EndLimit` is *not* needed to read `0x34`. It only governs whether the driver acts on a limit. It remains `Disable`.

13.4 Sensor characterisation

A magnet parked at the edge of the sensor's range makes the output oscillate across its threshold, which on a live limit would read as the endstop repeatedly opening and closing. Held deliberately, both channels are clean: the longest uninterrupted hold measured **5.27 s with zero bounce**. Short events and one 0.09 s re-trigger were traced to the magnet physically slipping, confirmed by the operator, not to an electrical fault.

13.5 Motion: Joint C drives its load, contradicting 2026-08-25

With coils enabled in `SR_vFOC`, a commanded 4° joint move and return:

	Encoder	From start
start	2270	—
after 4° out	14621	+4.002°
after return	2271	+0.0003°

Commanded 4.000° against 4.002° measured, and a return error of **one encoder count**. Motion was smooth and monotonic throughout. A subsequent 25° leg at speed 15 was equally exact (+25.003°).

This supersedes the 2026-08-25 conclusion (Section 1) that Joint C had an open path between the driver's output stage and the coils. It does not.

Caution: `0xFD` pulses are 3200/motor-rev, not 16384 encoder counts

The `POSITION_CONTROL` pulses field is in microsteps (200 full steps × 16), while the encoder reports 16384 counts/rev. Confusing the two is a 16384/3200 = 5.12× magnitude error. For a 67.82:1 joint, N joint-degrees is round($N \times 67.82/360 \times 3200$) pulses.

13.6 The run that failed, and why

A 25° return leg stalled partway, leaving the joint at +16.48°. A retry at speed 30 behaved worse: two legs did not move at all and a third travelled fast and uncommanded. The driver then stopped answering the bus entirely.

The cause was neither electrical nor a protocol fault — **the motor had overheated and its power was switched off**. The link statistics showed zero bus-errors and the CANable was still enumerated, which is the signature of a powered-down node rather than a bus problem.

Caution: Heat accumulates across trials, not within one

This run stacked a ∼40 s move directly onto a three-leg retry with no cooling gap. No single move was aggressive; the accumulated duty cycle was. This is the same lesson recorded for Joint B — space motion trials out and keep total energised time short, rather than judging each trial on its own.

13.7 New and changed scripts

- `joint_endstop_test.py` — read-only live monitor of the IO byte; prints only on change.
- `joint_endstop_dwell.py` **(new)** — timestamps every edge and reports how long each trigger actually held, with a SOLID/CHATTER/MARGINAL verdict. Written because the monitor above prints only on change and carries no timestamps, so a 50 ms tap and a two-second hold are indistinguishable in its output.
- `joint_io_diff.py` — sweeps every documented read-only register with the magnet off and on, for when the endstop does not appear on `0x34` at all.
- `joint_magnet_probe.py` — checks whether a magnet disturbs the encoder.

Caution: Reply frames are DLC 3; a DLC 2 frame is somebody's query

`joint_endstop_dwell.py` filters on `len(data) == 3`. The older scripts accept `>= 2`, so when two tools poll `can0` at once each can mistake the other's *query* frame for a driver *reply*. This briefly corrupted a reading during this session. Run one poller at a time until the older scripts are tightened.

13.8 Direction byte mapping, confirmed by eye

Measured after the motor had cooled and the driver had been power-cycled, using a slow 5° move at speed 5 in each direction with the rotation watched at the joint itself:

Direction byte	Encoder	Physical rotation
`0x00`	counts up	counter-clockwise
`0x80`	counts down	**clockwise**

Clockwise is `0x80`. Note that the 4° and 25° runs recorded above used `0x00` for their first leg, so they travelled *counter-clockwise* first, not clockwise — the encoder sign alone had been taken as "forward" without anyone checking the joint.

Motion at speed 5 was smooth and linear, about 700 counts per half-second with no roughness, and both legs settled at exactly ±5.005°: 15448 counts out against 15447 back, a one-count difference.

Caution: The vantage point was not recorded

"Clockwise" for a wrist joint depends on which side it is viewed from, and the observer's position was not written down at the time. Before this mapping is relied on for anything that matters, confirm it again and record the vantage with it.

13.9 Next steps

- Let the motor cool, then confirm Joint C still answers and still drives before anything else.
- Apply the DLC-3 reply filter to `joint_endstop_test.py` and `joint_io_diff.py`.
- Only then enable `EndLimit` and attempt `GoHome`, with the belt off: the manual states the shaft *unlocks* when homing reaches the left limit.
- Update `motor_driver.cpp`'s IO comment, which records the pre-remap resting byte `0x01` and is now misleading.

Appendices — standing reference

The material below is procedural rather than historical: it does not change from session to session, and it is kept here so the log itself stays chronological.

A Introduction & Core Concepts

Welcome to the Arctos Robotic Arm Integration Guide! This Standard Operating Procedure (SOP) will teach you how to wire, program, and control an industrial-style robotic arm using a communication framework called a **CAN Bus** (Controller Area Network) wrapped inside **ROS 2** (Robot Operating System).

Before writing code, there are two engineering concepts you must understand to prevent damaging the hardware:

A.1 The Power of Mechanical Advantage (Gearboxes)

The motors inside our robot do not drive the arm joints directly. Instead, they spin through a heavy reduction gearbox with a **67.82:1 ratio**.

- This means the inner motor shaft must spin completely around **67.82 times** just to rotate the physical outer robot arm joint by **1 single turn (**360°**)**.
- Because of this massive scaling, commanding the motor to spin a small distance results in tiny, highly precise shifts on the arm axis.

A.2 Closed-Loop Control & Artificial Stalls

Our motors use MKS Servo42D closed-loop controllers. Traditional stepper motors blindly take steps without knowing if they actually moved. Our closed-loop motors use a magnetic sensor on the back of the shaft to read exact positioning.

The Safety Catch: If the robot arm hits an obstruction or the timing belt is pulled too tight, the motor shaft will fall slightly behind its electronic target. The driver board will instantly trigger a **Tracking Error Protection Stall**, locking the motor completely down to prevent drawing excessive current or breaking parts. **If your motor stops responding completely and goes limp, you must power-cycle its main 12V/24V power supply to clear the safety latch.**

B Safety First! Emergency Procedures

Robotic arms possess immense torque and can easily pinch fingers or crush components if commanded incorrectly.

1. **Keep Your Distance:** Never place your hands near the belts, pulleys, or joint linkages while the main power supply is active.
2. **The Kill Commands:** Keep a separate terminal window open at all times running `candump -tz can0` to watch live network traffic. If the arm moves unexpectedly, prepare to type or execute the following emergency stop frame immediately:

```
# Instant Emergency Stop (Cuts all holding torque) -- Joint B, CAN ID 0x05
cansend can0 005#F7FC

# Explicit Motor Coil Disable -- Joint B, CAN ID 0x05
cansend can0 005#F300F8
```

Listing 2: Emergency Stop Over CAN Command Line

Caution: Every frame needs its checksum byte, including the emergency stop

Earlier versions of this document sent `F7` and `F300` with no checksum byte at all. Reading `motor_driver.cpp`'s `stopMotor()` and `can_protocol.cpp`'s `sendFrame()` directly confirms the real driver **always** appends a checksum – `sendFrame()` has no special case for `EMERGENCY_STOP`. The checksum is `(CAN_ID + command_byte + ... + data_bytes) & 0xFF` as used everywhere else in this document. For Joint B (`0x05`): `0xF7+0x05=0xFC`, so the frame is `F7FC`; for `F3 00` (disable): `0xF3+0x00+0x05=0xF8`, so `F300F8`. **Recompute the checksum for whichever CAN ID you're actually targeting** – don't copy the Joint B byte values above for a different joint.

C Hardware Map & Network Setup

The communication backbone relies on the **CANable 2 USB Adapter**. This device translates serial data coming from our computer into high-speed physical differential voltage lines (`CAN_H` and `CAN_L`) that daisy-chain from motor to motor down the body of the robot chassis.

C.1 The Joint Architecture Matrix

Each joint on the network operates using a specific identifier (CAN ID). When sending commands over the bus, you must target these hex values exactly:

Robot Joint	Physical Role
Joint X	Base/waist rotation (first joint from `base_link`)
Joint Y	(role not confirmed in repo docs)
Joint Z	(role not confirmed in repo docs)
Joint A	Elbow
Joint B	Wrist Rotation
Joint C	Wrist joint (confirmed with the machine's owner, 2026-09-01 – consistent with this being the *la*

Table 2: Arctos Arm Network Node Mappings, per `motor_id` in `arctos_description/config/arctos_controller.yaml` – the real 6-joint arm configuration. **Confirm every value in this table empirically with `can_id_scan.py` before trusting it** – Joint C's CAN ID was listed as `0x03` in earlier drafts of this document and in other guide documents in this project (that value is actually Joint Z's ID in a separate, simplified 3-joint test-rig config, `arctos_moveit_base_xyz/config/ros2_controllers.yaml`), and Joint X's DIP switches turned out not to map to any CAN ID at all once it was discovered that driver was the RS485 hardware variant (Section E.1.1).

C.2 Bringing Up the SocketCAN Network (First Time SOP)

Every time you plug the CANable adapter into your Ubuntu computer, you must bridge the serial hardware interface to the Linux networking core. Follow these steps exactly:

```
# 1. Verify your computer sees the USB hardware device
lsusb | grep -i canable
# Expected: ID 16d0:117e CANable2

# 2. Identify the active serial port handle
ls -l /dev/ttyACM*
# Note down if it returns /dev/ttyACM0 or /dev/ttyACM1

# 3. Clean out old dead connections
sudo pkill slcand
sudo ip link delete can0 2>/dev/null

# 4. Bind the adapter to the Linux socket layers at 500kbit/s (-s6)
sudo slcand -o -c -s6 /dev/ttyACM0 can0
sudo ip link set can0 up

# 5. Check to confirm the network state is ACTIVE
ip -details -statistics link show can0
```

```
# Look for: <NOARP,UP,LOWER_UP> and state ERROR-ACTIVE
```

Listing 3: Linux Terminal CAN Interface Setup

C.3 Automated Setup Script

`scripts/setup_canable.sh` in the `ros2_arctos` repository automates the five steps above. **If you pulled this repo before August 2026, re-pull it first**: the script previously attached the adapter with `slcand -s3` (100 kbit/s) and then tried to override the bitrate with `ip link set can0 type can bitrate 500000` – a netlink attribute that native slcan interfaces do not support, so the override silently failed and the bus was left running at the wrong speed. It now attaches directly at `-s6` (500 kbit/s), matching the manual steps above.

```
cd ~/arctos_ws/src/ros2_arctos
chmod +x scripts/setup_canable.sh
sudo ./scripts/setup_canable.sh
```

Listing 4: Running the Automated Setup Script

D Protocol Errata: Reconciling This Project's Debugging Documents

This workspace accumulated several other debugging documents alongside this one – `arctos_CAN_ROS2_SOP_updated.tex`, `arctos_joint_b_guide.tex` (since removed — its content was consolidated into `arctos_motor_guide.tex`), and `arctos_motor_can_cheat_sheet_updated.md` – written at different points while reverse-engineering the same MKS CAN protocol. They do not fully agree with each other. Rather than silently pick one, this section cross-checks the disagreements directly against the real source (`motor_types.hpp`, `can_protocol.cpp`, `motor_driver.cpp`) and records the resolution here, since that source is the actual ground truth for what the C++ ROS 2 driver will do.

Caution: Encoder read opcode: use `0x31`, not `0x30`

`arctos_joint_b_guide.tex` (now removed; it was the older of the two documents) standardized on `0x30` as "the confirmed encoder read command," describing a carry+value response format, and treats `0x31` as unconfirmed. This does not hold up: `motor_types.hpp` defines `READ_ENCODER = 0x31` and no `0x30` command at all – the only place `0x30` appears anywhere in `arctos_motor_driver`'s source is as raw test-fixture bytes for an unrelated velocity-decoding unit test, not a CAN command opcode. `arctos_CAN_ROS2_SOP_updated.tex` reaches the correct conclusion (`0x31`, decoded via the 48-bit `decodeInt48()` format used throughout this document) by citing the same source files. **Use `0x31` with the 48-bit decode, as every script in this package already does.** Consequently, the old `mks_driver.py`'s `read_absolute_encoder()` (in both `arctos_can` and the early `arctos_can_control` draft), which sends `0x30`, is querying a command the real driver does not define and should not be trusted or built upon.

Caution: Checksum required on every frame, including disable and emergency stop

Several manually-run `cansend` examples across these documents (and in earlier drafts of this SOP, now fixed in Section 2) omit the checksum byte on `F3 00` (disable) and `F7` (emergency

stop). `can_protocol.cpp`'s `sendFrame()` has no special case for any command – it always appends `(motor_id + sum(data_bytes)) & 0xFF` as the last byte. Always compute and include it, as every script in this package's `send_frame()`/`checksum()` pair already does.

Caution: Working-mode mismatch can look like a communication failure

Both newly-added documents independently flag something not previously captured in this SOP: `motor_types.hpp` defines a six-way `MotorMode` enum (`CR_OPEN=0`, `CR_CLOSE=1`, `CR_vFOC=2`, `SR_OPEN=3`, `SR_CLOSE=4`, `SR_vFOC=5`), set via `SET_WORKING_MODE (0x82)`, and the ROS 2 driver defaults to assuming `working_mode=2` (`CR_vFOC`). If the physical driver's own panel/config is set to a different mode than whatever a script assumes, the controller can accept and checksum-acknowledge a motion frame (`0xF5`/`0xF4`) **without the shaft actually rotating** – which reads exactly like a wiring or CAN-ID problem but isn't one. If Step 3 of Section E sends a checksum-acknowledged move and the encoder genuinely does not change afterward (not just a small/ambiguous change – see the next caution), check the driver's working-mode setting before assuming the CAN link or the frame format is at fault.

Caution: The `-813 pulses/degree` constant and any `0x30`-based safety limit are not trustworthy

Consistent with Section E.1.1's and this section's other findings: the `joint_profiles` placeholder in `arctos_can/mks_driver.py` (`pulses_per_degree = -813.0`) does not match the source-verified ≈3086.56 counts/degree figure for the 67.82:1-geared joints (off by roughly 3.8×), and its accompanying "safety workspace cage" estimates the limit using a different multiplier (`-0.947`) than what is actually transmitted, so it cannot reliably predict or block an unsafe target. Do not reuse this constant or that limit-checking code for Joint C or any other 67.82:1 joint without first reconciling it against the gear-ratio math confirmed in this document.

E Bringing Up an Uncalibrated Joint (One at a Time)

This project's actual practice has been to wire up and bring up **one joint at a time** – first Joint X, then (after that driver turned out to be the wrong hardware variant, Section E.1.1) Joint C – rather than trust the full arm's wiring and configuration all at once. This section is written as a repeatable procedure for whichever joint is currently the only one wired to the bus: get trustworthy *information* out of it first, and only reach actual *motion* as a small, explicitly-confirmed last step. Table 3 gives the confirmed parameters for each joint bench-tested so far; substitute the relevant row into the generic commands below.

Neither CAN ID in the table above should be trusted without running Step 1 – Joint X's DIP switches (2 and 3 ON) turned out not to map to any working CAN ID at all once it was discovered that driver was the RS485 hardware variant, and Joint C's ID has disagreed across this project's own documents (`0x03` in an earlier hardware-map table and in other guide documents found in this workspace, versus `motor_id: 6` i.e. `0x06` in the authoritative `arctos_controller.yaml` – the `0x03` value actually belongs to a different joint, `Z_joint`, in a separate simplified 3-joint test-rig config).

Joint	CAN ID
X	`0x01` (per YAML; driver was actually RS485, Sec. E.1.1)
C	`0x06` per `arctos_controller.yaml`, `0x03` per older project docs – **found at `0x05` via `can_id_scan.py`**

Table 3: Joint instances bench-tested with this procedure so far. Both `inverted=true`, both `requires_homing=false` per `arctos_controller.yaml`. Encoder resolution is 16384 counts/motor-revolution for both.

Do not use the placeholder values in the `joint_profiles` dict of `arctos_can/mks_driver.py` either, which were copy-pasted from Joint B's calibration and are not valid for any other joint.

E.1 Step 1 – Confirm the Joint's Real CAN ID

Run the discovery script below. It only sends the read-only `0x31 READ_ENCODER` query (the same opcode the real C++ driver in `arctos_motor_driver` uses) to a range of candidate IDs and reports who answers. It never enables coils and cannot move the arm.

```
cd ~/arctos_ws
source install/setup.bash
python3 src/arctos_can_control/arctos_can_control/can_id_scan.py --channel can0 --
    ids 1-16
```

Listing 5: Confirming a Joint's CAN Address

Exactly one ID should respond, since only one joint should be wired at a time. Use that confirmed value – not the table above by assumption, and not a guess from the DIP switches – for every command in the rest of this section.

E.1.1 Troubleshooting: Zero Responses AND Zero Bus Errors

If the scan above (and a full baud-rate sweep across every standard rate, DIP switches aside) comes back completely silent **with the link statistics also showing zero bus-errors** (`ip -details -statistics link show can0`), stop tuning baud rates and DIP switches and check the driver's part number first. A real CAN node at the wrong bitrate still generates bus errors, because it is at least attempting to decode CAN framing and losing arbitration/ACK. Total silence with zero errors, despite confirmed 12–24V driver power and confirmed CAN_H/CAN_L continuity and termination, is the signature of a driver that has **no CAN transceiver at all** – most commonly because it is the **RS485 variant** of the MKS SERVO42D/57D rather than the CAN variant. RS485 and CAN both use a differential pair, but RS485 here carries MKS's own async serial command protocol, not CAN framing; a real CAN controller and an RS485 transceiver cannot talk to each other over the same wires no matter what bitrate or address is configured. If this happens, physically confirm the board's part number/silkscreen before spending more time on the software side, and if it is the RS485 variant, either exchange it for the CAN variant (keeps this entire document, the shared CAN daisy-chain, and every script in this package working unchanged) or treat that joint as a separate RS485 subsystem requiring its own USB-RS485 adapter and a driver written against MKS's RS485 protocol document, which is a different frame format and checksum from everything described here and is out of scope for this SOP. **This is exactly what happened with Joint X** – the driver installed there was the RS485 variant, which is why Joint C is now the joint under test instead.

E.2 Step 2 – Read-Only Reliability Test

Before commanding any motion, verify the CAN link to this joint is actually trustworthy: response rate, checksum integrity, and encoder jitter while the joint sits still. `joint_reliability_test.py` sends only `0x31` (encoder), `0x39` (error state), and `0x3A` (enable state) – it never sends `0xF3` (enable) or a position command, so coils are never energized and the motor cannot move as a result of running it.

```
# Joint C example -- substitute the confirmed CAN ID and correct gear ratio
# from Table 3 for whichever joint is actually wired up.
python3 src/arctos_can_control/arctos_can_control/joint_reliability_test.py \
    --channel can0 --can-id 0x06 --gear-ratio 67.82 --samples 200
```

Listing 6: Joint Reliability / Telemetry Test

The script reports a response rate, checksum failure count, round-trip latency, and stationary encoder jitter, then prints a plain-language verdict on whether the link is clean enough to move on to Step 3. Do not proceed to Step 3 until response rate is above 95% with zero checksum failures.

E.3 Step 3 – Supervised First-Motion Test

Run this yourself, standing next to the arm with a hand on the power switch. Do not run it unattended or from a remote session. If this is a joint's first-ever motion test, its coils have never been energized before and it has no calibrated travel limits yet, so nothing in software stops it from driving into a mechanical hard stop if a sign or scale assumption below turns out wrong.

```
# In a second terminal, keep this running the whole time:
candump -tz can0

# In your main terminal (Joint C example):
python3 src/arctos_can_control/arctos_can_control/joint_micro_move_test.py \
    --can-id 0x06 --gear-ratio 67.82
```

Listing 7: Supervised Joint Tiny-Move Test

The script prints the starting encoder position, then requires you to type `go` before it enables coils and commands a small (˜0.1–0.3° of joint travel) absolute move, printing live position every half second so you can watch for stall or runaway behavior. Ctrl+C at any point sends an immediate, checksummed `0xF7` emergency stop – the script prints the exact checksummed frame for whichever CAN ID you passed it at startup, so you can also trigger it manually from another terminal at any time, e.g. for Joint C (`0x06` once reprogrammed): `0xF7+0x01=0xF8`:

```
cansend can0 006#F7FD
```

Listing 8: Emergency Stop for Joint C

E.3.1 Troubleshooting: Enable Acknowledged, But No Holding Torque

Joint C's actual bench test hit this: `can_id_scan.py` found it at `0x05` (not `0x06` per `arctos_controller.yaml`, not `0x03` per older docs), Step 2's reliability test came back perfect (100% response rate, 0 checksum failures, 0 encoder jitter), and `ENABLE_MOTOR` was checksum-accepted with `READ_ENABLE_STATE (0x3A)` confirming a flip from disabled to enabled – but the shaft still spun completely freely by hand, with no holding torque at all, and a commanded `0xF5` move produced zero encoder change over several seconds.

The real `arctos_hardware_interface` source (`arctos_interface.cpp`'s `setupMotorParameters()`) sends `SET_WORKING_MODE (0x82)` with `MotorMode::SR_vFOC` (value `5`) to every joint before ever enabling it – something none of the raw Python scripts in this document had been doing. Sending that first (`cansend can0 001#820588` for CAN ID `0x01`: $0x82 + 0x05 + 0x01 = 0x88$) did make `READ_ENABLE_STATE` flip from `0` to `1` on the next enable – but the shaft was still free-spinning, meaning that CAN-reported flag does not by itself confirm the coils are actually driven. `motor_types.hpp` also defines `SET_CURRENT (0x83)` (payload: 16-bit little-endian mA) with a source default of `working_current{1600}`, and its setter is likewise never actually called anywhere in the real ROS 2 stack (the call site in `setupMotorParameters()` is commented out) – so it depends entirely on whatever the panel has stored. Explicitly setting it to this motor family's documented rated current (1200 mA) before enabling made no difference either.

At that point, two CAN-configurable causes had been ruled out with real test data, which shifts the likely cause to something CAN commands cannot fix remotely:

- **Motor phase wiring** – confirm all four phase wires are firmly seated between the driver's motor output terminals and the motor itself. A loose or disconnected phase wire lets the driver's internal state machine believe it is enabled and driving current, while delivering no actual torque.
- **Main motor power specifically to this driver** – confirm the 12–24V motor power rail (not just logic/CAN power) is actually connected to *this* driver board. A driver can respond to CAN reads/writes on logic power alone while having no motor-power rail connected at all.
- `SET_ENABLE_SETTINGS (0x85)` – defined in `motor_types.hpp` but its payload format is not documented anywhere in this project (no guide document, no source comment). Do not guess-write to this opcode; if the driver has its own physical screen/panel, check its enable-pin/enable-mode setting there directly instead.

E.4 Step 4 – Read-Only ROS 2 Telemetry

Once Steps 1–2 pass, `joint_state_publisher` exposes live joint position on `/joint_states` without any ability to command motion – it has no command subscriber by design, so it is safe to leave running continuously while you work on calibration.

```
cd ~/arctos_ws
colcon build --packages-select arctos_can_control
source install/setup.bash

# Terminal 1: telemetry-only node (Joint C example)
ros2 run arctos_can_control joint_state_publisher --ros-args \
    -p joint_name:=C_joint -p can_id:=6 -p gear_ratio:=67.82

# Terminal 2: watch live position
ros2 topic echo /joint_states
```

Listing 9: Joint ROS 2 Telemetry Node

Only after Step 3 has been repeated a few times with consistent, expected direction and magnitude – and real travel limits have been established, e.g. by wiring and testing the hall-effect home sensor per the `ros2_arctos` README – should a joint move on to closed-loop position commands from a higher-level controller.

F The Python Code Stack

To build an automated system, we separate our project into a clean library file (`mks_driver.py`) that handles the mathematical scaling transformations, and an execution test wrapper (`run_arm.py`).

F.1 The Core Driver: `mks_driver.py`

Caution: This listing is kept for historical reference – see Section D before using it

The `read_absolute_encoder()` method below sends opcode `0x30`, which `motor_types.hpp` does not define as a command at all (the real driver defines `READ_ENCODER = 0x31`); and the −813.0 pulses/degree figure below does not match the source-verified ≈3086.56 counts/degree for a 67.82:1-geared joint. Both are explained in Section D. Every script added later in this document (Section E) uses the corrected `0x31` opcode and does not use this pulses-per-degree constant.

Create a folder inside your workspace and save the following class module. This calculates our empirical real-world joint tracking scale of −813.0 **driver pulses per** 1° **of physical joint rotation**.

```
import can
import time

class MKSServoDriver:
    def __init__(self, channel="can0", interface="socketcan"):
        self.bus = can.interface.Bus(channel=channel, interface=interface)
        self.ENCODER_CPR = 16384

        # Unified Joint Profile Configuration Matrix
        self.joint_profiles = {
            0x05: {"pulses_per_degree": -813.0, "invert": False}, # Joint B
        }

    def checksum(self, motor_id, data_bytes):
        """MKS SUM-8 Checksum validation."""
        return (motor_id + sum(data_bytes)) & 0xFF

    def send_frame(self, motor_id, data_bytes):
        crc = self.checksum(motor_id, data_bytes)
        frame = data_bytes + [crc]
        msg = can.Message(arbitration_id=motor_id, data=frame, is_extended_id=
    False)
        self.bus.send(msg)
        return frame

    def set_motor_state(self, motor_id, enable=True):
        state_byte = 0x01 if enable else 0x00
        self.send_frame(motor_id, [0xF3, state_byte])
        time.sleep(0.1)

    def read_absolute_encoder(self, motor_id, timeout=1.0):
        self.send_frame(motor_id, [0x30])
        end = time.time() + timeout
        while time.time() < end:
            resp = self.bus.recv(timeout=end - time.time())
```

```
            if resp is not None and resp.arbitration_id == motor_id and resp.data[0] == 0x30:
                carry = int.from_bytes(resp.data[1:5], byteorder="big", signed=True)
                value = int.from_bytes(resp.data[5:7], byteorder="big", signed=False)
                return (carry * self.ENCODER_CPR) + value
        raise TimeoutError(f"Motor {motor_id} failed to return encoder data.")

    def move_relative_degrees(self, motor_id, degrees, speed=25, accel=5):
        current_encoder = self.read_absolute_encoder(motor_id)
        profile = self.joint_profiles[motor_id]

        target_pulses = int(degrees * profile["pulses_per_degree"])

        # Absolute Software Boundary Filters (Safety Workspace Cage)
        estimated_final_encoder = current_encoder + int(target_pulses * -0.947)
        MIN_LIMIT, MAX_LIMIT = -73000, 73000
        if estimated_final_encoder < MIN_LIMIT or estimated_final_encoder > MAX_LIMIT:
            print(f"Movement Blocked: Target {estimated_final_encoder} violates safety limits!")
            return

        pos = target_pulses & 0xFFFFFF
        data = [0xF4, (speed >> 8) & 0xFF, speed & 0xFF, accel, (pos >> 16) & 0xFF, (pos >> 8) & 0xFF, pos & 0xFF]
        self.send_frame(motor_id, data)

    def close(self):
        self.bus.shutdown()
```

Listing 10: mks_driver.py Module

F.2 The Target Runner: run_arm.py

This basic executable test sequence leverages our driver module to run a clean, safe, relative shift:

```
#!/usr/bin/env python3
from arctos_can.mks_driver import MKSServoDriver
import time

JOINT_B_ID = 0x05
arm = MKSServoDriver(channel="can0")

try:
    start_pos = arm.read_absolute_encoder(JOINT_B_ID)
    print(f"Current Position Baseline: {start_pos} encoder counts")

    print("Powering up motor coils...")
    arm.set_motor_state(JOINT_B_ID, enable=True)

    TARGET_ANGLE = 5.0
    print(f"Executing a large continuous sweep of +{TARGET_ANGLE} degrees...")
    arm.move_relative_degrees(JOINT_B_ID, degrees=TARGET_ANGLE, speed=60, accel=20)

    time.sleep(3.0) # Allow mechanism transit time settling
    end_pos = arm.read_absolute_encoder(JOINT_B_ID)
```

```
    print(f"End encoder: {end_pos} (Delta: {end_pos - start_pos})")

finally:
    print("Shutting down cleanly.")
    arm.set_motor_state(JOINT_B_ID, enable=False)
    arm.close()
```

Listing 11: run_arm.py Execution Script

F.3 Compiling and Launching over the ROS 2 Command Line

Make sure your workspace is built, sourced, and execute standard command-line tools to monitor parameters:

```
# 1. Compile your localized package source workspace
cd ~/arctos_ws
colcon build --packages-select arctos_can_control
source install/setup.bash

# 2. Run the main processing node thread
ros2 run arctos_can_control joint_control

# 3. Open a separate terminal tab to audit live incoming SI Radians telemetry
ros2 topic echo /joint_states

# 4. Open a third tab to inject a precise motion command into the active network stream
ros2 topic pub --once /joint_b_command std_msgs/msg/Float64 "{data: 5.0}"
```

Listing 12: Building and Testing over ROS 2 CLI